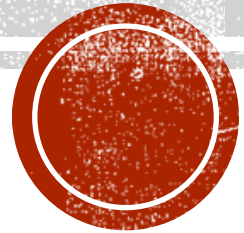
LECTURE 06

Flutter routes

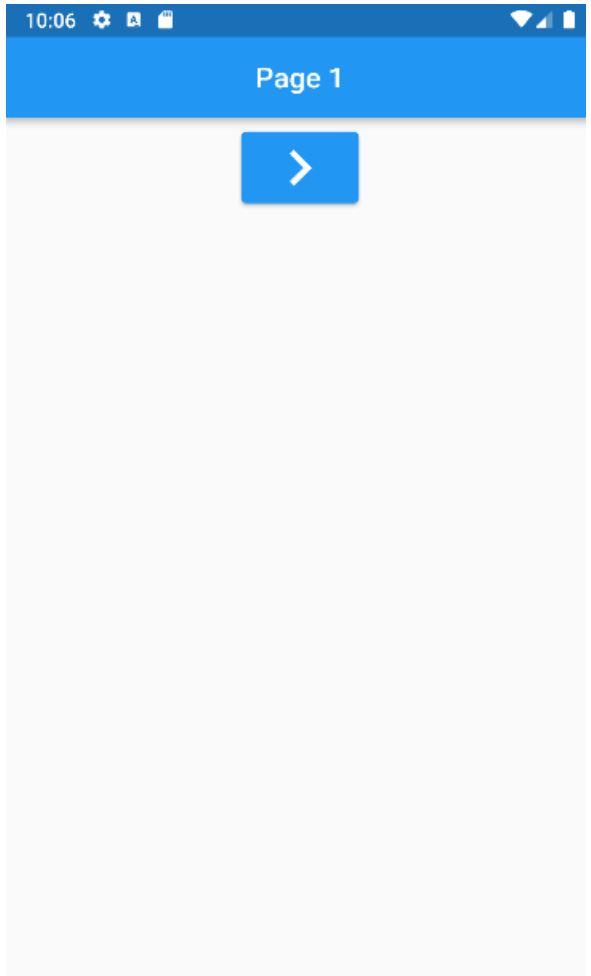


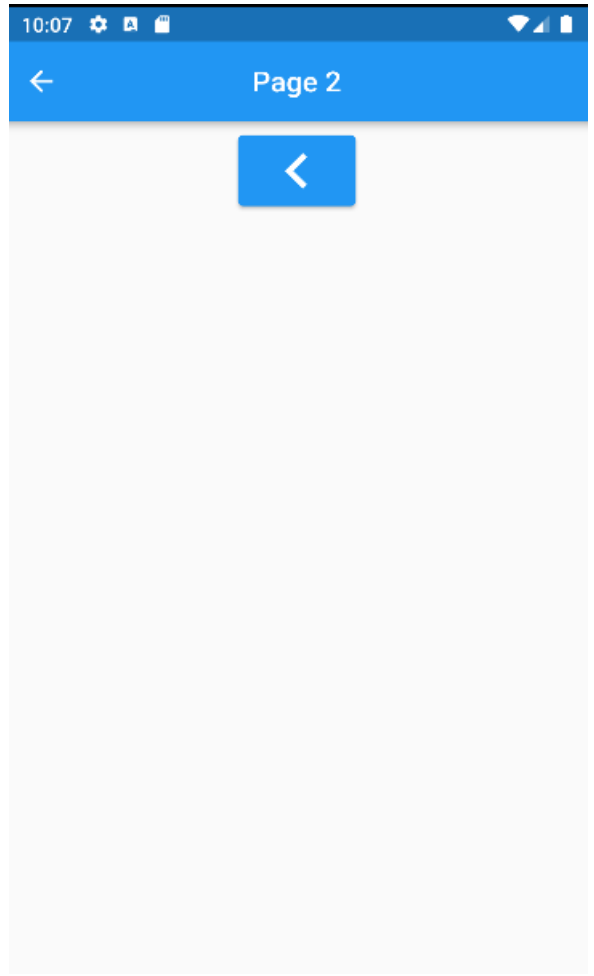
FLUTTER ROUTES

- We will discuss how to create multiple pages (screens) and navigate between them.
- Flutter uses objects called routes to navigate between pages.
- There are multiple methods for using routes, the simplest is the Navigator class.
- The Navigator class is a simple way to navigate between pages. It is best suited for mobile applications.
- The Navigator class does not work well with web-applications. Since it doesn’t support web-browser history and the browser’s forward button.

FIRST APPLICATION

- The first Application will include 2 pages.
- Each page has an AppBar and ElevatedButton with an Icon.
- The user can navigate between the 2 pages using the ElevatedButtons.





MAIN.DART CODE

```
import 'package:flutter/material.dart';
import 'page1.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'csci410 week 7 project',
      home: Page1(),
    );
  }
}
```

THE NAVIGATOR CLASS

- The Navigator class keeps track of pages (screens) using a stack.
- You open a new page by calling the push method and passing to it a route object. In this case we will pass to it a MaterialPageRoute object.
- The push method places the page on the top of the stack and then navigates (displays) the page.
- Pages are stacked one above the other in FILO fashion using a stack structure.
- We go back to the previous page by calling the Navigator's pop method. It removes the page at the top of the stack and displays the page beneath it.

THE NAVIGATOR PUSH FUNCTION

- The below code demonstrates how we can use the Navigator's push function to move from Page 1 to Page 2.

- Page 1 will remain active (running) in the background, while Page 2 is being displayed.

```
Navigator.of(context).push(
      MaterialPageRoute(builder: (context) => const Page2())
    );
```

WHAT ARE CONTEXTS IN FLUTTER?

- In Flutter, the context refers to the location of a widget in the widget tree. It provides information about the surrounding environment and services that the widget might need.

- The context can contain information such as the theme of the app, the locale, the screen size, and more. By accessing the context, a widget can make decisions about how to display itself based on the context.

- The MaterialPageRoutes object takes a callback function with a BuildContext object as parameter. You set the body of the function to call a widget, that will be displayed in a new page.

```
Navigator.of(context).push(
      MaterialPageRoute(builder: (context) => const Page2())
    );
```

FLUTTER ICONS

- Flutter has many predefined icons, that you can access using The Icons list.
- You create an icon by creating an Icon widget and passing to it an IconData object from the Icons list.
- You can also define a size for the icon, using the size field.
- Below we will create an ElevateButton with an icon as a child.

```
ElevatedButton(onPressed: () {}, // disabled button, no defined function
        child: const Icon(Icons.navigate_next, size: 50)
    )
```

PAGE1.DART CODE

```
import 'package:flutter/material.dart';
import 'page2.dart';

class Page1 extends StatefulWidget {
  const Page1({Key? key}) : super(key:
key);

  @override
  State<Page1> createState() =>
_Page1State();
}

class _Page1State extends State<Page1> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Page 1'),
        centerTitle: true,
      ),
body: Center(child: Column(children: [
        const SizedBox(height: 10),
        ElevatedButton(onPressed: () {
          Navigator.of(context).push(
            MaterialPageRoute(builder: (context) => const
Page2())
          );
        },
            child: const Icon(Icons.navigate_next, size: 50)
        )
      ]),
      ),
);
}
}
```

PAGE2.DART CODE

```
import 'package:flutter/material.dart';

class Page2 extends StatelessWidget {
  const Page2({Key? key}) : super(key:
key);

  @override
 Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(
       title: const Text('Page 2'),
       centerTitle: true,
     ),
body: Center(child: Column(children: [
       const SizedBox(height: 10),
       ElevatedButton(onPressed: () {
         Navigator.of(context).pop();
       },
          child: const Icon(Icons.navigate_before, size: 50)
       )
      ]),
      ),
      );
  }
}
```

HOW IT WORKS?

- Page 2 contains a StatelessWidget that displays an ElevatedButton, that allows the user to go back to Page 1.

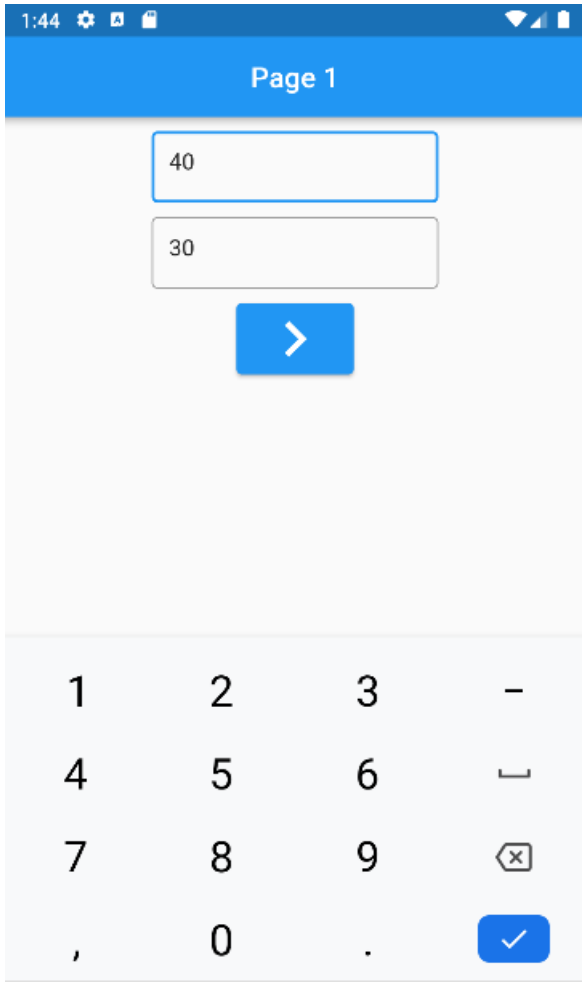
- We used a StatelessWidget, since the widget will be re-created every time we display the page. Its state will not change at runtime.

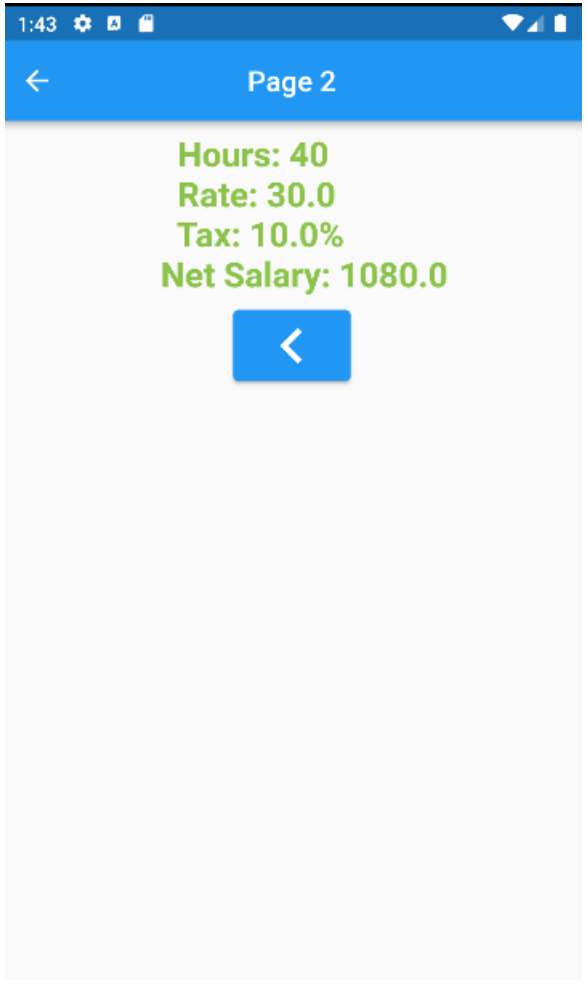
- We used the Navigator's pop function, to remove the page from the top of the stack and return to Page 1.

```
ElevatedButton(onPressed: () {
    Navigator.of(context).pop();
  },
    child: const Icon(Icons.navigate_before, size: 50)
  )
```

SECOND APPLICATION

- The second Application will allow the user to enter hours and rate and then compute an employee's net salary.
- It demonstrates how you can send data from one page to the other.





EMPLOYEE.DART CODE

```
/*
We will create a class to represent an
employee in the employee,dart file.
*/
class Employee {
  int _hours = 0;
  double _rate = 0;
  final double _tax = 0.1; // default 10% tax

  Employee(int hours, double rate) {
    if (hours <= 0 || rate <= 0) {
      throw Exception('negative values not
allowed');
    }
    _hours = hours;
    _rate = rate;
  }

  double netSalary() {
    return (_hours * _rate) * (1 - _tax);
  }

  @override
  String toString() {
    return """
      Hours: $_hours
      Rate: $_rate
      Tax: ${_tax * 100}%
    Net Salary: ${netSalary()} """;
  }
}
```

THE TEXTEDITINGCONTROLLER

- We will use the TextEditingController, to retrieve the value of the TextFields.

- We will declare a TextEditingController for each TextField in page1.dart, as shown in the below code:

```
class _Page1State extends State<Page1> {
  final TextEditingController _controllerHours = TextEditingController();
  final TextEditingController _controllerRate = TextEditingController();
```

ADDING A DISPOSE DESTRUCTOR

- Dispose is a method triggered whenever the created object from the stateful widget is removed permanently from the widget tree. It is generally overridden and called only when the state object is destroyed.

- We will use it to dispose of the TextEDitingControllers when its not used anymore. Its always good practice to override this method in such cases.

```
@override
 void dispose() {
  _controllerRate.dispose();
  _controllerHours.dispose();
  super.dispose();
 }
```

THE OPENPAGE2 FUNCTION

- We will add a function called openPage2 to the _Page1State class.

- The function will be called by the ElevatedButton.

```
void openPage2() {
  try {
   int hours = int.parse(_controllerHours.text);
   double rate = double.parse(_controllerRate.text);
   Navigator.of(context).push(
      MaterialPageRoute(builder: (context) => const Page2(),
        // we send data using the settings and pass to it an Employee object
         settings: RouteSettings(arguments: Employee(hours, rate))
      )
   );
  }
  catch(e) {
   print(e); // better to remove print in release version
  }
}
```

ADDING THE TEXTFIELDS

Now we will add the TextFields to the body of the Scaffold in the _Page1State class. Observe how we set the controller property of the TextField.

```
body: Center(child: Column(children: [
      const SizedBox(height: 10),
      SizedBox(width: 200, height: 50,
      child: TextField(controller: _controllerHours, keyboardType: TextInputType.number,
         decoration: const InputDecoration(border: OutlineInputBorder(), hintText: 'Enter hours')),
      ),
      const SizedBox(height: 10),
      SizedBox(width: 200, height: 50,
      child: TextField(controller: _controllerRate, keyboardType: TextInputType.number,
         decoration: const InputDecoration(border: OutlineInputBorder(), hintText: 'Enter rate')),
      ),
      const SizedBox(height: 10),
      ElevatedButton(onPressed: openPage2,
         child: const Icon(Icons.navigate_next, size: 50)
      )
    ]),
    ),
```

RETRIEVING THE EMPLOYEE DATA

Now we will modify page2.dart, to retrieve the employee object send from Page 1. We will use the ModalRoute class, to retrieve a reference to the Employee object created in Page 1.

```
class Page2 extends StatelessWidget {
  const Page2({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final employee = ModalRoute.of(context)!.settings.arguments as Employee;
```

ADDING TEXT WIDGET TO PAGE 2

Now we will modify the return of the body of the Scaffold in Page 2, to add a Text widget, that will display the employee information. Notice how we call employee.toString.

```
body: Center(child: Column(children: [
      const SizedBox(height: 10),
      Text(employee.toString(), style: const TextStyle(color: Colors.lightGreen, fontSize: 24, fontWeight: FontWeight.bold)),
      const SizedBox(height: 10),
      ElevatedButton(onPressed: () {
        Navigator.of(context).pop();
      },
         child: const Icon(Icons.navigate_before, size: 50)
      )
    ]),
    ),
```